

Firestore CRUD Operations

Simple FooBar Application

Database Programming Course

Educational Demo with Dart and Firestore

Learning Objectives

After this lesson, you will understand:

-  **CRUD Operations** in Firebase/Firestore
-  **Data Modeling** with Dart classes
-  **Serialization** for database storage
-  **Error Handling** in database operations
-  **Testing** database applications
-  **Project Structure** for maintainable code



Project Structure

```
foobar/  
├── lib/  
│   ├── models/  
│   │   └── foobar.dart           # Data model  
│   ├── services/  
│   │   └── foobar_crud_firebase.dart # CRUD operations  
│   ├── main.dart                 # Demo application  
│   └── firebase-config.json      # Firebase configuration  
├── test/                         # Unit tests  
├── doc/                          # Documentation  
└── pubspec.yaml                  # Dependencies
```

Simple, organized, and educational!

Data Model: FooBar Class

```
class FooBar {  
    final String? id;      // Document ID  
    final String foo;      // String field  
    final int bar;         // Integer field  
  
    FooBar({  
        this.id,  
        required this.foo,  
        required this.bar,  
    });  
}
```

Key Concepts:

- **Simple Structure:** Only 2 data fields
- **Nullable ID:** For new documents
- **Required Fields:** Ensures data integrity
- **Immutable:** Final fields for safety

Serialization Methods

To Firebase (Map):

```
Map<String, dynamic> toMap() {  
  return {  
    'foo': foo,  
    'bar': bar,  
  };  
}
```

From Firebase (Map):

```
static FooBar fromMap(  
  Map<String, dynamic> map,  
  [String? documentId]  
) {  
  return FooBar(  
    id: documentId,  
    foo: map['foo'] ?? '',  
    bar: map['bar'] ?? 0,  
  );  
}
```

Why Serialization?

Firebase stores data as JSON-like documents, not Dart objects!

Firebase CRUD Service

```
class FooBarCrudFirebase {  
    late Firestore _firestore;  
    final String _collectionName = 'foobars';  
  
    // Initialize connection  
    Future<void> initialize({String projectId = 'foobar-a1317'}) async {  
        Firestore.initialize(projectId);  
        _firestore = Firestore.instance;  
    }  
  
    // CRUD methods...  
}
```

Design Principles:

- **Single Responsibility:** Only handles database operations
- **Clear Naming:** Methods match CRUD operations

+ CREATE Operation

```
Future<FooBar?> create(FooBar foobar) async {  
  try {  
    // Add document to collection  
    Document doc = await _firestore  
      .collection(_collectionName)  
      .add(foobar.toMap());  
  
    // Return with new ID  
    return foobar.copyWith(id: doc.id);  
  } catch (e) {  
    print('✗ Error creating: $e');  
    return null;  
  }  
}
```

What Happens:

1. Convert FooBar → Map
2. Send to Firebase collection
3. Firebase generates unique ID
4. Return FooBar with new ID

Error Handling:

- Returns `null` on failure
- Logs error for debugging

READ Operations

Single Document:

```
Future<FooBar?> read(String id) async {
  try {
    Document doc = await _firestore
      .collection(_collectionName)
      .document(id)
      .get();

    return FooBar.fromMap(doc.map, doc.id);
  } catch (e) {
    return null;
  }
}
```

All Documents:

```
Future<List<FooBar>> readAll() async {
  try {
    List<Document> docs = await _firestore
      .collection(_collectionName)
      .get();

    return docs.map((doc) =>
      FooBar.fromMap(doc.map, doc.id)
    ).toList();
  } catch (e) {
    return [];
  }
}
```

QUERY Operations

```
Future<List<FooBar>> readByBar(int barValue) async {  
  try {  
    List<Document> docs = await _firestore  
      .collection(_collectionName)  
      .where('bar', isEqualTo: barValue) // Filter condition  
      .get();  
  
    return docs.map((doc) => FooBar.fromMap(doc.map, doc.id)).toList();  
  } catch (e) {  
    return [];  
  }  
}
```

Firestore Query Features:

- `where()` - Filter documents
- `orderBy()` - Sort results

UPDATE Operations

Full Update:

```
Future<bool> update(
    String id,
    FooBar updatedFoobar
) async {
    try {
        await _firestore
            .collection(_collectionName)
            .document(id)
            .update(updatedFoobar.toMap());

        return true;
    } catch (e) {
        return false;
    }
}
```

Partial Update:

```
Future<bool> updateFields(
    String id,
    Map<String, dynamic> updates
) async {
    try {
        await _firestore
            .collection(_collectionName)
            .document(id)
            .update(updates);

        return true;
    } catch (e) {
        return false;
    }
}
```

DELETE Operation

```
Future<bool> delete(String id) async {  
  try {  
    await _firestore  
      .collection(_collectionName)  
      .document(id)  
      .delete();  
  
    return true;  
  } catch (e) {  
    print('✗ Error deleting: $e');  
    return false;  
  }  
}
```

Important Notes:

- **Permanent:** Deleted data cannot be recovered

- **Cascading:** Does not delete related documents

Main Application Demo

```
Future<void> main() async {
  final crudService = FooBarCrudFirebase();

  try {
    await crudService.initialize();

    // CREATE
    FooBar newFooBar = generateRandomFooBar();
    FooBar? created = await crudService.create(newFooBar);

    // READ
    List<FooBar> all = await crudService.readAll();

    // UPDATE
    await crudService.update(created!.id!, updatedFooBar);

    // DELETE
    await crudService.delete(created.id!);

  } finally {
    crudService.close();
  }
}
```

Testing Strategy

Model Tests:

```
test('should create FooBar', () {  
  final foobar = FooBar(  
    foo: 'test',  
    bar: 42  
  );  
  
  expect(foobar.foo, equals('test'));  
  expect(foobar.bar, equals(42));  
});
```

Service Tests:

```
test('should handle CRUD workflow', () {  
  // Test with mock Firebase  
  // or integration testing  
  
  // CREATE → READ → UPDATE → DELETE  
});
```

Testing Levels:

- **Unit Tests:** Models and logic
- **Integration Tests:** Firebase operations
- **End-to-End Tests:** Complete workflows

Running the Application

```
# Install dependencies
dart pub get

# Run tests
dart test

# Run the demo
dart run lib/main.dart

# Use the run script (with educational output)
./run.sh
```

Expected Output:

```
 Starting Firebase FooBar CRUD Demo
 STEP 1: CREATE Operations
Created: FooBar{id: abc123, foo: hello, bar: 42}
 STEP 2: READ Operations
```

Key Learning Points

What We Learned:

1.  **Structure:** Organized code with models and services
2.  **Modeling:** Simple, focused data classes
3.  **Serialization:** Converting between objects and Firestore format
4.  **CRUD:** All basic database operations
5.  **Error Handling:** Graceful failure management
6.  **Testing:** Comprehensive test coverage
7.  **Documentation:** Clear explanations and examples

Next Steps

For Students:

1.  **Experiment:** Modify the FooBar model
2.  **Extend:** Add more fields or methods
3.  **Query:** Try different Firebase queries
4.  **Test:** Write more comprehensive tests
5.  **Deploy:** Set up your own Firebase project
6.  **Build:** Create a Flutter UI for this backend

Advanced Topics:

- **Security Rules** in Firebase
- **Real-time Listeners** with Streams

Resources

Documentation:

- **Firestore:** <https://firebase.google.com/docs>
- **Firestore:** https://pub.dev/packages/firebase_firestore
- **Dart Testing:** <https://dart.dev/guides/testing>

This Project:

- **Source Code:** Complete working example
- **Tests:** Comprehensive test suite
- **Documentation:** This presentation + code comments

 **Thank You!**

Questions?

Remember:

- Start simple, then add complexity
- Test your code thoroughly
- Document your learning journey
- Practice with real Firebase projects

Happy Coding! 